
CYCLES · DEEP EXR · VFX PIPELINE

# MattRM2 VFX Build

## Documentation — based on Blender 5.1.2

A custom Blender build for professional VFX pipelines. This document covers the production features added on top of official Blender 5.1.2, plus the included bug fixes.

Created by Matthieu Barbié · 2026

v0.7 — Beta

# 0 Table of Contents

# 1 Deep EXR — Cycles CPU

Industry-standard deep compositing, Arnold / RenderMan architecture

This build adds a full **Deep EXR** implementation to Cycles, following the **Arnold / RenderMan architecture**. Deep EXR is the industry-standard format for high-quality depth compositing — it stores multiple colour and depth samples per pixel, which lets a compositor merge elements by true scene depth instead of a flat 2D over.

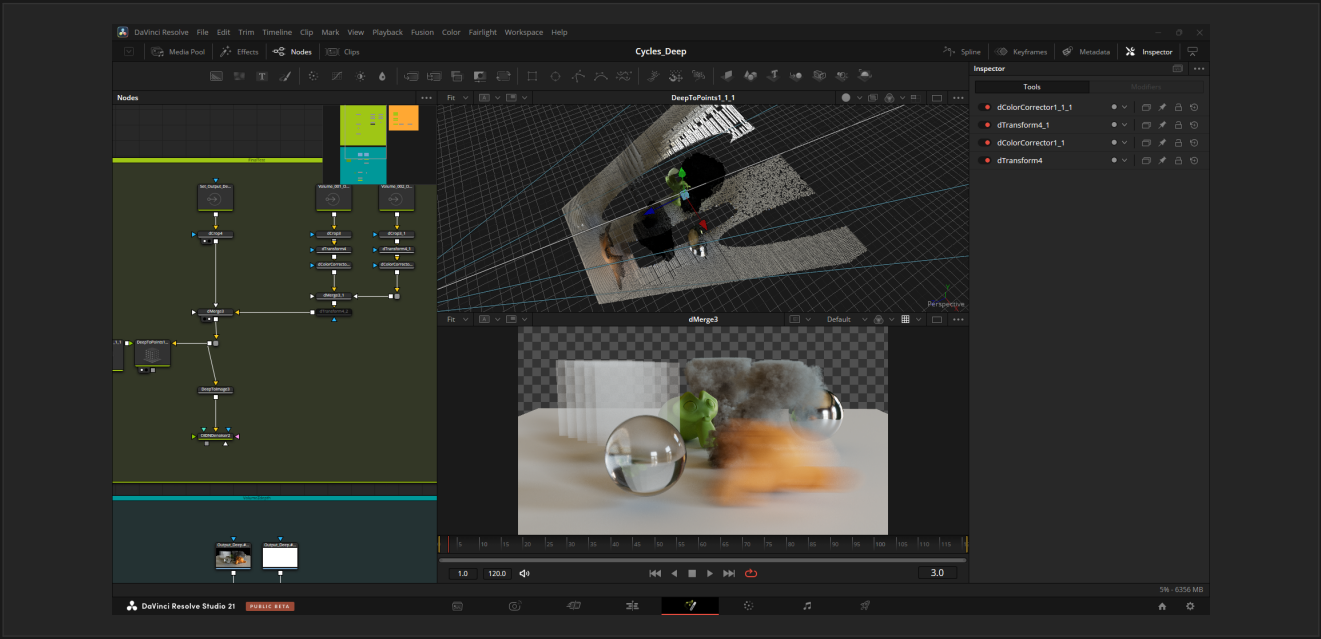



*Fig. 1.1 — Cycles Deep EXR composited in DaVinci Resolve — Fusion*

## 1.1 Overview

### Key characteristics:

- **Surfaces** — recorded per sample, so Motion Blur and Depth of Field are preserved naturally. The deep flatten is pixel-perfect identical to the standard flat EXR render.
- **Volumes** — Arnold-style fixed-step single-ray raymarch per pixel, decoupled from the path-traced volume integration. Multiple overlapping/intersecting volumes on the same view layer are fully supported.
- **Output** — standard OpenEXR deep scanline format, readable by Nuke, Fusion, Resolve, and any OpenEXR-compliant compositor.
- **Per-view-layer output** — each view layer gets its own subfolder and filename prefix automatically.

> **Pixel-perfect flatten**
>
> *The deep flatten matches the beauty render exactly: you can drop the deep EXR into your comp and get the same image as the flat render, then gain depth-based merging on top.*

## 1.2 Panel Reference — View Layer > Passes > Deep EXR

All Deep EXR settings live in **View Layer Properties > Passes > Deep EXR**. Each view layer has its own independent settings.

| Setting | Description |
|---|---|
| **Enable** | Activates Deep EXR output for this view layer. |
| **Max Depth Layers** | Maximum surface fragments per pixel (-1 = unlimited). |
| **Max Volume Depth Layers** | Maximum volume slabs per pixel (-1 = unlimited). |
| **Z Threshold** | Minimum Z distance to merge nearby surface fragments (scene units). |
| **Volume Step Size** | Raymarch step size for volume slabs (scene units). Match it to your volume scale — e.g. 0.01 m works well for smoke at 1 m scale. |
| **Transparent vs World** | How transparent / alpha surfaces interact with the world background in the deep. |
| **Cache Path** | Temporary directory for the per-tile deep fragment cache during rendering. Fragments are streamed to disk after each tile to keep RAM usage flat regardless of scene complexity. A local SSD is strongly recommended. |
| **Color Depth** | Float Full (32-bit) or Float Half (16-bit) per channel. |
| **Codec** | EXR compression for the deep file. Deep OpenEXR only allows ZIPS (default, recommended), RLE, or None. |
| **Override Output Path** | Custom output path template (supports // and #### frame notation). |

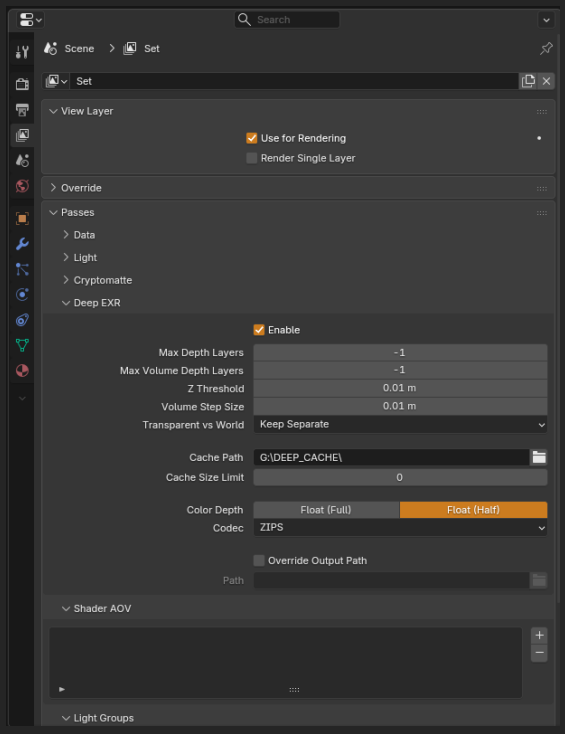



*Fig. 1.2 — The Deep EXR panel in View Layer Properties*

## 1.3 Quick Start

Follow these steps to produce your first deep EXR:

| | |
|---|---|
| 1 | Set the render engine to **Cycles** and the volume integration to **Unbiased** (Render Properties > Volumes). |
| 2 | In **View Layer Properties > Passes > Deep EXR**, check **Enable**. |
| 3 | Set a **Cache Path** (e.g. "C:/DEEP_CACHE/" or "//DEEP_CACHE") — it must be writable; a local SSD is recommended. |
| 4 | Adjust **Volume Step Size** to match your scene scale. |
| 5 | Set your output path, or use **Override Output Path** for a custom location. |
| 6 | Render — the deep EXR is written after the render completes. |

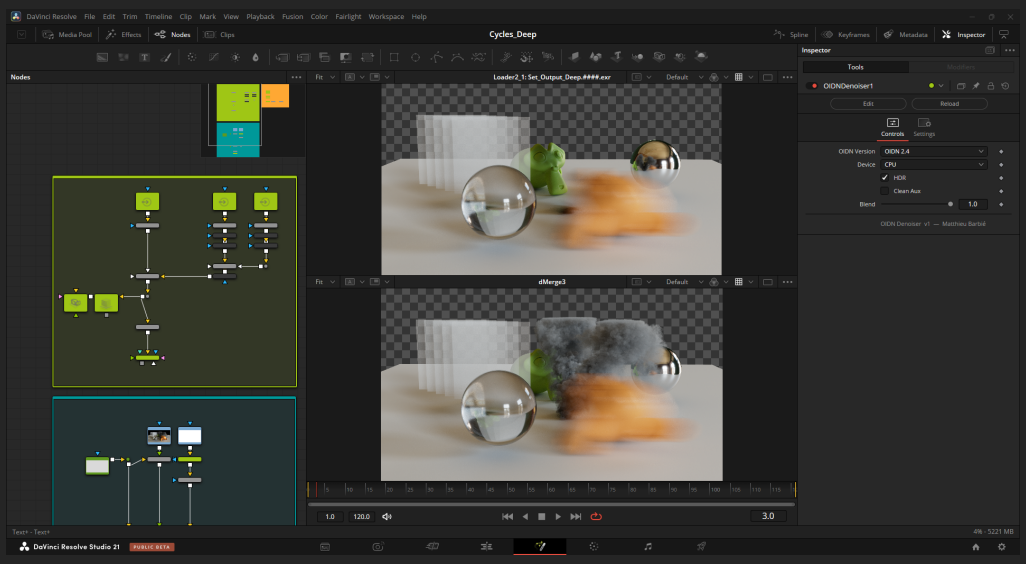


*Fig. 1.3 — Basic deep compositing from the resulting deep EXR*

## 1.4 Output Structure

Each view layer is written to its own subfolder with its own filename prefix, so multi-layer setups stay organised automatically:

```
/
  Deep/
    View_Layer/
      View_Layer_output.0001.exr
    FX_Layer/
      FX_Layer_output.0001.exr
```

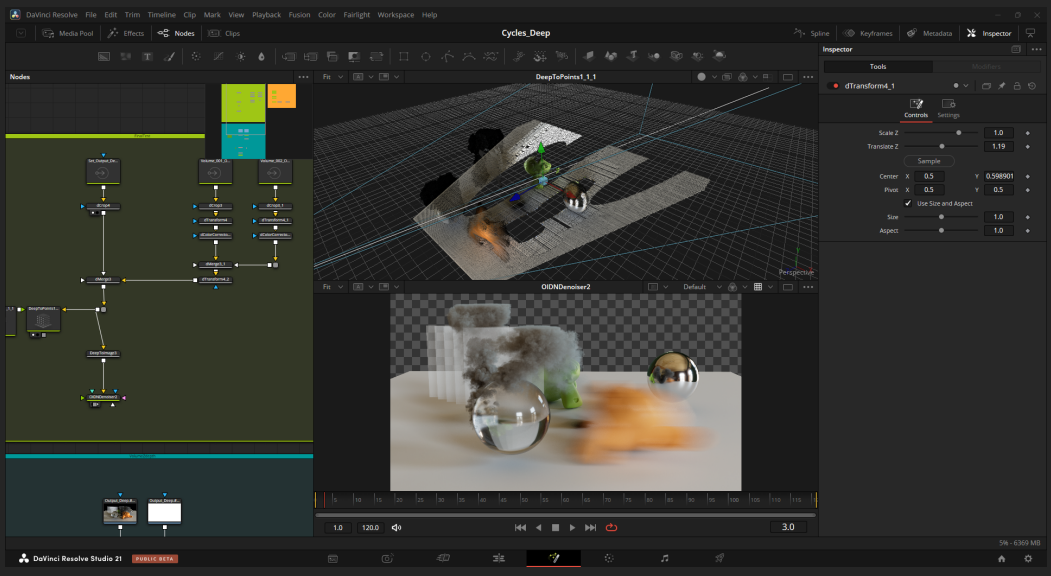


*Fig. 1.4 — Deep EXR depth-based compositing in Fusion*

## 1.5 Volume Limitations

**Unbiased mode only.** Deep EXR volume output requires the Cycles volume integration mode to be set to **Unbiased** (delta-tracking). The Biased mode (ray-marching) produces darker, heavier output with no visual gain for deep compositing, and introduces voxel artefacts.

**Unbiased required for volumes**
*A red warning is shown in the panel if both Deep EXR and Biased volume are active at the same time. Switch the volume integration to Unbiased before rendering deep volumes.*

**CPU only in this release**
*GPU (OptiX / CUDA) deep rendering is on the roadmap. This release is CPU only.*

## 1.6 Multi-Layer Deep EXR Output

A dedicated **output format** that writes every view layer's deep data into a single multi-part OpenEXR — **one deep part per view layer** — instead of one deep file per layer in subfolders. Select it in **Output Properties > Media Type > Multi-Layer Deep EXR**, right next to *Multi-Layer EXR*.

- **One deep part per layer** — each view layer becomes its own deep part (RGBA + Z + ZBack), named after the layer, following the Nuke one-part-per-layer convention.
- **A true deep file** — the file contains only deep parts, so Nuke and Fusion read it directly as deep (no flat fallback). The beauty is the deep flatten (DeepToImage).
- **Single compression** — one codec (None / ZIPS / RLE) for the whole file, set in the Output panel; these are the only codecs valid for deep data.
- **Automatic** — selecting the format enables deep recording for the render.

**Deep file vs AOVs**
*Need the flat passes / AOVs (Diffuse, Normal, Cryptomatte…) too? Render them in parallel as a standard Multi-Layer EXR — the usual VFX split of a deep file and an AOV/beauty render. Choosing any non-deep format keeps the per-layer behaviour from section 1.4 (flat render plus per-layer deep files in Deep/<layer>/ subfolders).*

**Roadmap — deep AOVs / DeepID**
*Embedding the passes inside the deep (per-fragment Diffuse, IDs…) requires per-fragment recording — that is the upcoming DeepID feature. This deep-only format is the foundation it will build on.*

# 2 Z-Depth Pass

Antialiased, volume-aware and transparency-aware depth

The standard Z depth pass is upgraded to an **antialiased, volume-aware and transparency-aware** depth. There is no mode selector — just enable **Depth**, and it works with or without Deep EXR.

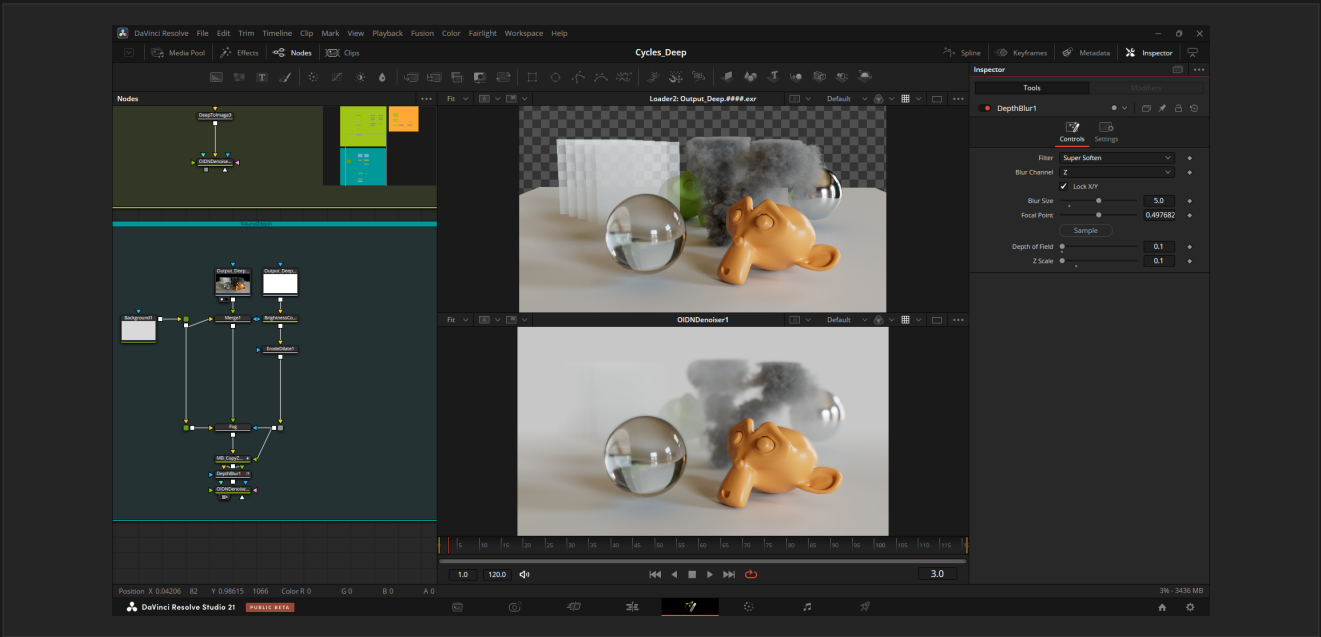



*Fig. 2.1 — New Z-Depth with volumes, viewed in DaVinci Resolve — Fusion*

## 2.1 Overview

The final **Depth** channel is a **min(surface, volume)** combine — *nearest wins*, like a compositor "Darken". This is immune to the over-bright that an alpha-over depth produces under motion blur and depth of field.

**Difference vs stock Blender**

*This diverges from stock Blender's Z (non-antialiased, single sample, 1e10 on empty pixels). The output here is a true antialiased depth, ready for Normalize, defocus, and depth-driven atmospherics.*

**CPU, CUDA & OptiX — SVM and OSL**

*The antialiased volume-aware Z-Depth runs on CPU, CUDA and OptiX, with both SVM and OSL volume shaders on GPU. On OptiX, OSL volume shading is resolved through the OSL shade pipeline, so the volume depth is computed natively (no surface-only fallback).*

## 2.2 What the Pass Does

- **Antialiased surface depth** — accumulated per sample and coverage-weighted, so edges are smooth instead of the hard, aliased single-sample Z of stock Blender. The value is the true (unpremultiplied) scene depth.
- **Volume aware** — volumes contribute a deterministic per-pixel depth (5 jittered sub-rays for clean silhouettes), composited so the nearest of surface / volume wins. Multiple and overlapping volumes are handled correctly.
- **Opacity-weighted volume depth** — low-opacity volumes (wispy smoke, thin atmosphere) are blended proportionally toward the far background depth rather than reading at their full near depth. A fully opaque volume reads at its true near depth; a nearly transparent volume reads near the background.
- **Transparency aware** — semi-transparent surfaces (and antialiased edges) over the background read at a weighted depth blended toward the far background, so a depth-driven fog / atmosphere shows correctly through them.
- **Clean background** — empty / transparent pixels are filled with the maximum captured scene depth, so a Normalize node maps the whole frame cleanly (no 1e10 spike on empty pixels like stock Blender).

## 2.3 Limitations

**Motion blur over volumes.** A motion-blurred surface partially in front of a volume produces a hard depth edge at the motion-blur boundary (the per-pixel volume depth is static, so it cannot fade per sub-sample with the blurred surface).

> **Workaround**
> *Render the surface and the volume on two separate View Layers, each with its own Depth pass, then combine them with a Math > Minimum node in the compositor. Each pass is then computed in its own clean context and the motion blur stays correct.*

**Surface / volume boundary.** A thin gap or hard edge may be visible at the contact line between a surface and an adjacent volume. A one-pixel **Erode / Dilate** in the compositor is enough to clean it up in most cases. The two-View-Layer workaround above also eliminates it.

# 3 OSL GPU Group-Data Budget

Raise the per-shader OSL group-data limit on GPU

Open Shading Language shaders allocate a fixed per-thread *group-data* buffer on GPU, capped at **2048 bytes** in stock Blender. Heavy custom OSL graphs can exceed it and fail to load on GPU with a "group data size" error, with no way to raise the limit from the interface.

## 3.1 Overview

This build adds an **OSL GPU Group Data** selector in **Render Properties**, shown when **Open Shading Language** is enabled (on CPU or OptiX). The budget ranges from **2048** (default) up to **6144** bytes, in 1024-byte steps. Raising it lets heavier custom OSL graphs compile and render on GPU.

## 3.2 Trade-off & Constraints

**Trade-off**

*The buffer is reserved per-thread, so a larger budget lowers GPU occupancy and slows down all OSL shading in the render — the impact can be significant. It only affects GPU rendering; OSL on CPU is unlimited. Changing the value recompiles the OSL kernels.*

If a shader still exceeds the highest budget, simplify the OSL graph (fewer layers, break up node groups, drop large arrays) or render that shader on CPU, where group data is unlimited. The **CYCLES_OSL_GROUPDATA_ALLOC** environment variable overrides the menu for advanced setups.

# 4 Environment Variables in File Paths

Portable, pipeline-friendly path resolution

File paths anywhere in Blender — render output, textures, libraries, caches, and even Preferences — now support **environment variable expansion**. This makes .blend files portable across machines and render farms with different local configurations.

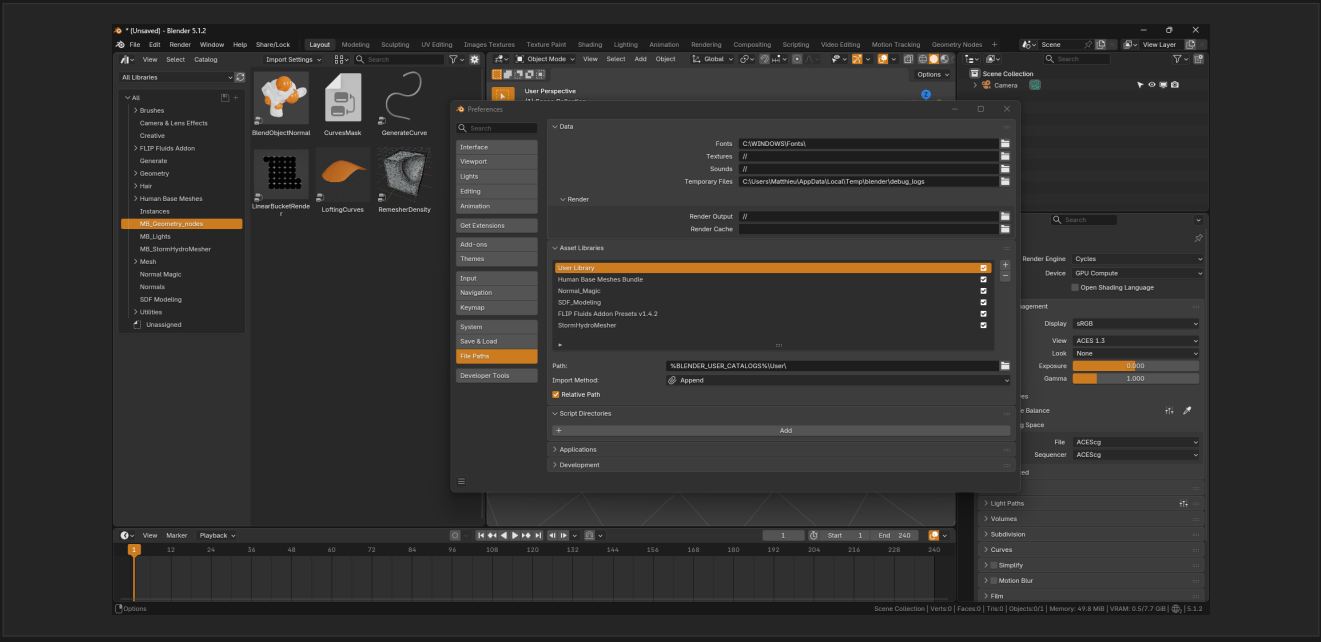


*Fig. 4.1 — Full environment-variable support across Blender file paths*

## 4.1 Overview & Supported Syntaxes

All three common syntaxes are supported, cross-platform:

```
$MY_PROJECT/renders/####.exr
${SHOT_DIR}/textures/diffuse.png
%PIPELINE_ROOT%/assets/char.blend
```

Variables are expanded at **path resolution time**, not at load time, so a .blend file stays portable: the same scene resolves to the right local paths on each artist's machine and on the farm, driven entirely by the environment.

> **Tip**
>
> *Combine environment variables with Blender's // relative-path notation and #### frame padding for fully pipeline-driven, machine-independent output paths.*

# 5 Qt Tools (PySide6)

An extensible, separate-process Qt UI framework for pipeline tools

This build bundles a small framework to run rich **Qt (PySide6)** user interfaces for pipeline tools — asset browsers, farm submitters, advanced control panels — beyond what the native Blender UI allows.

## 5.1 Overview

- **Separate process** — each Qt window runs in its own process, fully isolated from Blender. Dragging or resizing the Blender window never freezes the tool (and vice-versa).
- **Always on top** — tool windows stay above Blender so they are never lost behind it.
- **Live two-way sync** — editing a value in the tool updates Blender instantly; changes made in Blender flow back to the tool.
- **Auto-loaded bridge** — the headless **MattRM2 Qt Bridge** add-on loads automatically; it relays data between Blender and the tools and has no UI of its own.

## 5.2 Enabling the Deep EXR Demo

A Deep EXR control panel ships as a demo of the framework. To use it:

1. In **Preferences > Add-ons**, enable **MattRM2 — Deep EXR Panel (Qt demo)** (search "MattRM2").
2. In the 3D Viewport, open the **N-panel** and go to the **MattRM2 Qt** tab.
3. Click **Open Deep EXR Panel** — the Qt window opens, live-bound to the Deep EXR render settings.

## 5.3 Building a Tool — for Developers

A control-panel tool is essentially a layout of Blender property paths. The shared SDK builds the right widget per type and handles all the IPC and theming:

```
from mattrm2_qt_sdk import run_control_panel

run_control_panel("My Tool", layout=[
    ("Output", [
        ("Codec", "scene.render.deep_exr_output_compression"),
    ]),
])
```

### Architecture:

• **Bridge** (auto-loaded, Blender side) — a stable public API: *launch_tool()*, *register_handler()*, *register_tools_dir()*. It resolves context-relative RNA paths (e.g. *scene.render.codec*) and relays get/set.

• **SDK** (Qt side) — the shared charter: Matt Dark theme, always-on-top window, auto-built widgets from the schema, two-way sync. Tools import it.

• **Where code lives** — UI and third-party integrations (render farm, asset / production trackers) live in the Qt tool process (its own Python, free to add dependencies); custom Blender-side logic is registered as a handler on the bridge.

> **License**
> *Qt for Python (PySide6) and Qt are bundled under the GNU LGPL v3 (dynamically linked). The license texts are in the distribution's licenses/ folder.*

# 6 Included Bug Fixes

Upstream Cycles / UI fixes shipped ahead of official patches

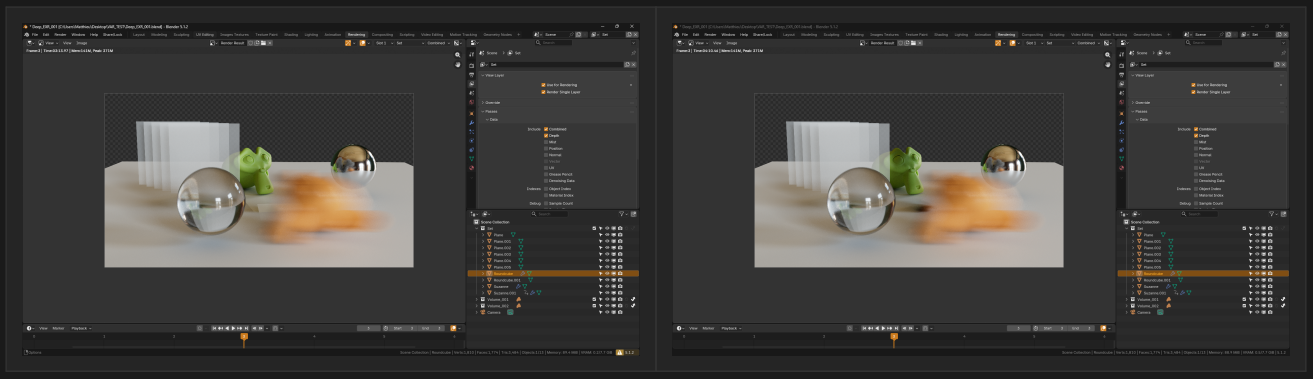
## 6.1 Node Editor Click-Drag

Fixed an official Blender 5.1 bug where click-dragging a node would move a different node than the one under the cursor. The incorrect node selection persisted in the .blend file. This fix is included ahead of the upstream patch.

## 6.2 Volume Indirect-Only Shadows

Fixed a Cycles bug where volumes placed in **Indirect-Only** collections would not cast shadows on surrounding surfaces.

In particular, when the volume's bounding box intersected the floor or another object, the shadow disappeared on the intersecting surface in this mode (most visible in Unbiased mode). This build renders the shadow correctly even with large bounding-box intersections.



**Before** — *stock Blender 5.1.2: no shadow where the volume's bounding box meets the floor*

**After** — *this build: shadow cast correctly, even on large bounding-box intersections*

*Fig. 6.1 — Volume Indirect-Only shadow, before / after the fix.*

> **Note**
> *The underlying issue is still present in official Blender 5.1.2 and will be reported upstream. This build ships the fix already applied.*

# 7 Known Limitations & Roadmap

Current constraints and what's coming next

## 7.1 Known Limitations

| Limitation | Notes / Workaround |
|---|---|
| **Deep EXR is CPU only** | GPU (OptiX / CUDA) port is on the roadmap. |
| **Deep EXR volumes require Unbiased mode** | Use Unbiased (delta-tracking). A warning is shown in the panel if Biased is active. |
| **Heavy OSL graphs may exceed the GPU group-data budget** | Raise the OSL GPU Group Data budget in Render Properties (up to 6144), simplify the shader, or render it on CPU (unlimited). |
| **Z-Depth: motion blur over volumes** | Hard depth edge where a motion-blurred surface is partially in front of a volume. Render surface and volume on separate View Layers, each with a Depth pass, and min() them in compositing. |
| **Multi-Layer Deep EXR + multi-view** | Cycles deep is mono (single view). The combined deep output is not supported with stereo / multi-view — render mono, or use the per-layer deep files. |

## 7.2 Roadmap

### Near Term

- **Deep EXR GPU** — port the deep volume raymarch and surface recording to OptiX / CUDA (currently CPU only). The antialiased volume-aware Z-Depth — now including OptiX + OSL — and the Indirect-Only volume fix already run on CPU, CUDA and OptiX.

### Medium Term

- **View Layer Attribute Overrides** — per-layer render settings, engine, samples, denoise and material overrides (Maya-style).
- **DeepID** — extend deep channels with objectId, materialId, normal and albedo per fragment.
- **Deep + DeepID Compositor Nodes** — native Blender compositing nodes for deep data manipulation (Deep Merge, Hold-Out, ID filter).
- **LPE (Light Path Expressions)** — custom AOVs via light path expressions (Arnold / RenderMan parity).

### Long Term

- **Plugins for DaVinci Resolve Fusion** — DeepID Sampler, Deep Fog, Deep Relight (Nuke-workflow parity in Fusion — the 'Deep+ Tools' for Fusion).
- **Caustics** — fast, accurate caustics (Photon Map / guided raymarcher — R&D).
- **Fix Unbiased volume limitation** — deep working with biased volumes.

- **R&D for heavy production scenes.**

**Feedback**

*This build is in active development. If you encounter unexpected behaviour, crashes, or rendering differences vs. stock Blender, please report with a .blend or minimal reproduction scene, your render settings, and your OS / GPU. Your feedback directly influences what gets built next. — Matthieu Barbié*

**License & Trademark**

*MattRM2 VFX Build is an independent build based on Blender 5.1.2, distributed under the GNU GPL v3. It also bundles Qt for Python (PySide6) and Qt under the GNU LGPL v3 (dynamically linked); the license texts are in the distribution's licenses/ folder. Not created, sponsored, or endorsed by the Blender Foundation. "Blender" is a trademark of the Blender Foundation. — blender.org*